

IT HAPPENS @ RAALE #4

Sentry on the Diff

Build a GitHub Action That Reviews C# Pull Requests for SOLID, Null-Safety and Async Correctness

DIFF → LLM → INLINE REVIEW

STUDENT HANDOUT

PART 0 — WHAT THIS PROJECT ACTUALLY IS

0.1 The problem in plain language

A C# team opens a pull request. Somewhere in that diff is a method that swallows a `NullReferenceException` waiting to happen, an `async void` that should be `async Task`, or a class that has quietly grown four responsibilities since the last review. A senior engineer would catch these in thirty seconds of reading — but senior engineers are busy, PRs pile up, and small smells get merged because nobody had the thirty seconds.

Right now this is manual. A human opens the “Files changed” tab, reads line by line, and leaves comments by hand — or, more often, doesn’t, and the smell ships.

Your job: build a GitHub Action that triggers on every pull request, reads only what changed, has a language model review that diff specifically for **SOLID violations, null-handling gaps, and async/await correctness**, and posts the findings back as **inline PR review comments** — with nothing manual in between.

A diff is not “the whole file” and it is not “free text” — it’s a structured, line-addressable artifact. That’s why this lands as **inline PR comments anchored to exact lines**, not a wall of text in the PR description. GitHub’s Review API is the conveyor belt between “the model found something” and “a human sees it exactly where it matters.”

0.2 What makes this different

Most student CI exercises end like this: write a script, run it once against a fixture, print some output, close the terminal. That is homework.

This project does not end there. You must deliver:

- **A running Action.** Someone must be able to open a real pull request against a real repo, and — with nobody from your team present — see review comments appear on the correct lines, automatically, on every push.
- **Something another team can drop into their own repo**, with one `uses:` line and one secret, without phoning you for help. This is why the project ships as a reusable composite/Docker Action, not a one-off script.
- **Proof that it works.** You must answer “did it actually review the diff, or did it hallucinate plausible-sounding comments?” with evidence, not a feeling.

There are two classic ways to fail this:

- A reviewer that comments on **whole files** instead of the diff is noisy and untrustworthy — it will flag pre-existing code the author didn’t even touch, and the PR author will start ignoring it.
- A reviewer that produces **confident, generic advice** (“consider adding null checks”) with no line anchor and no reference to the actual code is a linter cosplaying as a code reviewer. That is worse than useless — it trains the team to skim past your bot’s comments entirely.

You are marked on the whole path, PR event to inline comment.

PART 1 — IMPORTANT: THE REVIEW MUST BE GROUNDED

⚠ READ THIS SECTION CAREFULLY — teams lose marks every time by getting this wrong.

Your reviewer can be as simple as a rules-based static check for the three categories, or — if your organisers permit an LLM/API key — a language model prompted with the actual diff hunk to produce findings. Either approach is acceptable.

What is not acceptable is a comment that could have been generated without reading the diff at all — generic advice, invented line numbers, or a finding that references code that isn’t actually in the changed hunk.

Why this matters

- **Trust** — a wrong-but-confident inline comment is worse than no comment, because a reviewer approving the PR will trust it.
- **Verifiability** — anyone reading the PR must be able to check the comment against the actual diff line it’s anchored to.
- **It costs nothing to skip a file** — if a changed file has no findings, silence is the correct output, not a manufactured nitpick to look busy.

- **The point of the exercise** — wiring `pull_request` → an LLM call → a comment is a five-minute task. Grounding every finding in a real line of the real diff, and correctly anchoring it via the GitHub Review API's `line/side/commit_id` fields, is the skill being taught.

If you use a language model anywhere in the reviewer, you must be able to show, for each posted comment: the file, the exact diff hunk it was derived from, and the category (SOLID / null-handling / async) it falls under. A comment with no traceable hunk scores zero on the grounding component.

Ask your organisers whether an LLM or API key is permitted at this event before assuming you can call one.

PART 2 — BEFORE THE SESSION

You will not be given time to debug GitHub auth or Actions plumbing during the build. Arrive ready.

2.1 Set up a scratch repo and credentials

- A throwaway public repo you control, with C# source in it (any small ASP.NET/console solution works).
- A **fine-grained PAT** or a **GitHub App installation token** with `pull-requests: write` and `contents: read`. Prefer `GITHUB_TOKEN` (auto-provided in Actions) if its default permissions are enough — no need to manage a separate secret if so.
- If an LLM is permitted: an API key stored as a repo secret, never in code.

2.2 Pre-pull what you'll need

- Confirm `actions/checkout@vX` and `actions/github-script@vX` (or your language's GitHub SDK — Octokit for Node, or `Microsoft.NET` + `Octokit.NET` if you build the reviewer in C# itself) work in a throwaway workflow before the event.
- If self-hosting a Docker Action: pull your base image (`mcr.microsoft.com/dotnet/sdk:8.0` or similar) at home.

2.3 Bring test PRs

Prepare three branches at home so you're not scrambling on the night:

- A **small, clean diff** (one method, one obvious null-check gap) to sanity-check the pipe end to end.
- A **larger diff** (several files, a mix of real issues and clean code) to see how your reviewer behaves at realistic volume.
- A **deliberately unreviewable diff** — a generated file, a binary, a huge minified file — to test that your Action skips it politely instead of crashing or burning your entire token budget on it.

2.4 A stack that fits the clock

You may build this in any language the Action runner supports. One combination known to finish inside three hours:

Layer	Suggested	Note
Trigger	<code>on: pull_request: [opened, synchronize]</code>	<code>synchronize</code> is what fires on new pushes to an open PR
Diff source	<code>GET /repos/{owner}/{repo}/pulls/{pr}/files</code>	Gives you the patch (unified diff) per file directly — don't shell out to <code>git diff</code> unless you need more context
Runtime	Node 20 (<code>actions/github-script</code>) or a small C# console app	Node + Octokit is the path of least resistance for the GitHub API side; a C# analyzer step can do static SOLID/null/async checks natively via Roslyn if you want zero-LLM credit
Review posting	<code>POST /repos/{owner}/{repo}/pulls/{pr}/reviews</code> with a <code>comments[]</code> array	This is what makes comments land inline , not as a single PR-level comment
Static layer (optional, strong for grounding)	Roslyn analyzers / <code>dotnet build</code> with nullable warnings enabled	Free, deterministic findings for null-handling and some async misuse — use as ground truth the LLM's findings should agree with
LLM layer (if permitted)	One call per changed file's patch, structured JSON output	Prompt only with the patch hunk, not the whole file, to force grounding

Three notes worth knowing before you start:

- The `/pulls/{pr}/files` patch is truncated for very large files — check the `patch` field exists before parsing it, and skip (don't crash on) files where it's missing.
- Line numbers in a unified diff are **not** the same as line numbers in the final file — you must map hunk-relative positions to actual file line numbers before calling the Review API, or your comments will land on the wrong line.
- `pull_request` events from forks run with **read-only** `GITHUB_TOKEN` by default — decide early whether you're testing same-repo PRs (simpler) or need `pull_request_target` (more permissions, more risk) and say which you chose in the report.

PART 3 — WHAT YOU ARE BUILDING

Three moving parts, one workflow file.

GitHub PR event (opened / synchronize)

↓
.github/workflows/pr-sentry.yml

fetch-diff
→ changed hunks only

GET /pulls/{pr}/files

↓
reviewer
→ findings: {file,
line, category,
message}

static analyzer and/or LLM call
per changed hunk, category-tagged

↓
publisher
→ inline comments

POST /pulls/{pr}/reviews
event: COMMENT (never auto-approve/reject)

1. **fetch-diff** — pulls the PR's changed files and patches, discards binaries/generated files, and maps patch positions to real file lines.
2. **reviewer** — for each surviving hunk, checks for SOLID smells (God classes, mixed responsibilities, tight coupling), null-handling gaps (missing null checks, unguarded dereferences, `!` null-forgiving operator overuse), and async correctness (`async void`, missing `await`, blocking `.Result/.Wait()` calls). Every finding carries the file, line, and category.
3. **publisher** — posts one PR review with all findings as inline comments in a single API call, so the PR gets one clean review event rather than a flood of individual comments.

Why fetch the diff instead of reviewing whole files?

- It keeps every finding traceable to something the author actually touched, which is the whole grounding requirement in Part 1.
- It keeps token/API cost proportional to the size of the change, not the size of the repo.
- It avoids re-flagging pre-existing issues the PR didn't introduce, which is the fastest way to get a bot muted by a team.

PART 4 — THE INTERFACE CONTRACT

Your Action's internal steps should agree on these shapes so the pieces compose cleanly.

Findings (reviewer → publisher)

```
[
  {
    "file": "src/Billing/InvoiceService.cs",
    "line": 42,
    "category": "null-handling",
    "severity": "warning",
    "message": "customer.Address is dereferenced without a null check;
GetCustomer() can return null on a cache miss.",
    "hunk_ref": "@@ -38,6 +38,9 @@ public Invoice Create(..."
```

Posted review (publisher → GitHub Review API)

```
{
  "event": "COMMENT",
  "body": "PR Sentry found 4 items across 2 files (2 null-handling, 1 async, 1
SRP). See inline notes.",
  "comments": [
    {
      "path": "src/Billing/InvoiceService.cs",
      "line": 42,
      "side": "RIGHT",
      "body": "***[null-handling]** `customer.Address` is dereferenced without a
null check - `GetCustomer()` can return null on a cache miss."
    }
  ]
}
```

`event` must always be `COMMENT`, never `APPROVE` or `REQUEST_CHANGES` — the bot flags, a human decides. If no findings survive review, post nothing (or a single “no issues found in this diff” summary comment) rather than fabricating a finding to justify the run.

PART 5 — THE NO-DUPLICATE-COMMENTS RULE (MANDATORY)

Because `synchronize` fires on **every push** to an open PR, a naive Action will re-post the same findings on every commit, burying the thread.

Before posting, fetch existing review comments via `GET /pulls/{pr}/comments`, and only post a finding if no existing comment on that `(file, line, category)` already says the same thing. Either:

- **Skip** findings that already have a live comment, or
- **Resolve/update** the prior comment if the underlying code changed but the issue persists.

⚠ NO PARTIAL CREDIT ON THIS ONE Two pushes to the same PR with no code change between them must not double the comment count. Duplicate comments on a second run scores zero on the reproducibility component of Part 11 — no partial credit.

PART 6 — HOW TO ACTUALLY DO THIS

Read this whole part before coding.

6.1 Keep the finding categories boring

Three categories — SOLID, null-handling, async — each with a short, fixed list of patterns you actually detect (e.g. null-handling: unguarded dereference after a nullable-returning call; async: `async void`, blocking `.Result`). You are marked on grounded, correctly-anchored comments, not on catching every possible smell. If you finish early, widen the pattern list.

6.2 Build the pipe before the logic

This is the single most important paragraph in the document. The most common way to fail is to have a clever prompt at hour 2 and no workflow that successfully posts even one inline comment. Build the plumbing with fake logic first:

- workflow triggers on `pull_request` and checks out the code
- fetch-diff step fetches real files-changed but the reviewer step returns one **hardcoded** dummy finding on line 1 of the first changed file
- publisher step posts that dummy finding as a real inline comment

Once you see a real inline comment appear on a real PR from a real workflow run, replace the dummy finding with real logic. From that moment on, every minute you spend improves something that already works.

6.3 “The workflow ran” is not “the comment landed”

A green checkmark on the Action only means your steps didn't throw. It does not mean the comment is anchored to the right line — GitHub will silently reject a review comment whose `line/side/commit_id` don't correspond to a real diff position, and the API call can 422 without failing your job if you don't check the response. Check the response status of the `POST /reviews` call explicitly and log it.

6.4 Keep credentials out of the repo

Any API key or PAT goes into repo/organization **secrets**, referenced as `${{ secrets.NAME }}` in the workflow YAML, never hard-coded. Be ready to explain why: a key committed to a public repo's history is compromised the moment it's pushed, full stop — rotation doesn't undo the exposure window.

6.5 Line-mapping will lie to you if you let it

The unified diff's `@@ -a,b +c,d @@` header gives you the starting line in old and new file — you must walk the hunk to compute the real new-file line number for each `+` line before you can post a comment against it. Don't assume the patch's local line count equals the file's line count.

6.6 Make it repeatable

A second `pull_request: synchronize` event on an unchanged diff must produce the same (zero additional) comment count. Judges will push an empty commit and check nothing new appears.

6.7 Assume hostile input

Before the freeze, throw these at your Action and make sure it responds sensibly instead of crashing or spamming:

- a PR that touches only non-C# files (README, YAML)
- a PR that adds a huge generated/minified file
- a PR that deletes a file (no new lines to anchor comments to)
- a PR with zero net changes (e.g. a revert)
- an LLM call that times out or returns malformed JSON (if using an LLM)

A clean “no findings” or a skipped file is a pass. A failed workflow run or a duplicated comment thread is not.

PART 7 — REQUIREMENTS

7.1 Must-have — this is your pass

#	Requirement
1	The Action triggers on <code>pull_request: [opened, synchronize]</code> with zero manual steps
2	Only changed hunks are reviewed — the Action never re-flags code outside the diff

#	Requirement
3	Findings are posted as inline review comments via the Review API, correctly anchored to file + line
4	Every posted comment states its category (SOLID / null-handling / async)
5	The review event is always <code>COMMENT</code> , never <code>APPROVE/REQUEST_CHANGES</code>
6	A second push with no code change produces zero duplicate comments
7	Non-C# and unparseable/binary files are skipped, not crashed on
8	Secrets/tokens are read from Actions secrets, never hard-coded
9	The workflow fails loudly (non-zero exit) if the review-posting API call fails
10	Two identical PRs (same diff, different branch) produce the same finding set

7.2 Stretch — this is how you win

- A summary comment at the top of the review totalling findings by category
- Findings resolved/updated automatically once the flagged line is fixed in a later push
- A Roslyn-analyzer static layer used as ground truth the LLM's findings are cross-checked against
- p95 time from `pull_request` event to posted review under 60 seconds for a 500-line diff, actually measured
- Works correctly on fork PRs (`pull_request_target` handled safely, with the security trade-off documented)
- Composite Action published so another team can consume it with a single `uses:` line and zero copy-pasted YAML

PART 8 — TIMELINE (3 HOURS)

Time	Phase	You are done when
0:00 – 0:10	Read, agree architecture, split work	Everyone knows what they own
0:10 – 0:35		

Time	Phase	You are done when
	Skeleton — workflow + dummy finding end to end	A real PR gets one real (dummy) inline comment
0:35 – 1:05	Real fetch-diff — patches pulled, line-mapping correct	A <code>console.log</code> /debug dump shows correct file+line pairs for a real diff
1:05 – 1:45	Real reviewer — static and/or LLM logic for all 3 categories	Findings array has real, categorized, traceable entries
1:45 – 2:15	Real publisher — inline comments, no-duplicate rule	Two pushes to the same PR don't double comments
2:15 – 2:35	Hardening — hostile input, non-root/secrets hygiene, category tagging	The Part 10 checklist passes
2:35 – 2:40	BUILD FREEZE — commit and push	Nothing further is edited
2:40 – 3:00	Report	<code>REPORT.md</code> committed

Nominate one person as timekeeper and have them call out each checkpoint aloud.

🕒 THE 2:40 FREEZE IS HARD Work that is not committed does not exist.

PART 9 — THE REPORT (20 MINUTES)

One file: `REPORT.md`, in your repository.

1. **What we built** — five sentences plus the architecture. State honestly what works and what doesn't.
2. **The detection logic** — for each of the three categories, exactly what pattern(s) you detect and how (regex/AST/Roslyn/LLM prompt).
3. **Methods** — one line per row: diff source chosen vs rejected; line-mapping approach; duplicate-comment strategy; static vs LLM vs hybrid, and why.
4. **Results** — a table of at least six real PRs you tested against, what was flagged, whether it was correct, and — most important — your explanation of any false positive or false negative.
5. **How we worked** — planned vs actual at each Part 8 checkpoint, one dead end you abandoned and why.
6. **Limitations and next steps** — be specific. “Handle fork PRs securely” earns more than “add more checks.”
7. **How to run it** — exact steps to add this Action to a fresh repo, from this section alone.

PART 10 — PRE-FREEZE CHECKLIST

At 2:30, stop adding features and verify every line.

- ☐ Workflow triggers correctly on `opened` and `synchronize`
- ☐ Only changed-file hunks are reviewed, never whole files
- ☐ Comments land on the correct file + line (verified against a real PR, not assumed)
- ☐ `event: COMMENT` only — never auto-approve or auto-reject
- ☐ A second push with no diff change adds zero new comments
- ☐ Non-C#, binary, and generated files are skipped without crashing
- ☐ Secrets come from Actions secrets, not hard-coded
- ☐ Every finding is tagged with its category
- ☐ `REPORT.md` committed
- ☐ Everything pushed

PART 11 — MARKING

Component	Marks
Workflow triggers and runs clean, first time, no manual steps	15
Correct architecture — diff-only review, not whole-file	10
Comments correctly anchored (file + real line, not hunk-relative)	15
No-duplicate-comment rule on repeated pushes	10
Category coverage — SOLID, null-handling, async all represented	10
	15

Component	Marks
Grounding — every finding traceable to a real diff hunk	
Hostile-input handling	5
Report: methods and decisions	10
Report: results interpretation	6
Report: process and honesty	4
Total	100

A team with a static-analyzer-only reviewer, correct line-anchoring and zero duplicates will beat a team with an LLM-powered reviewer that posts comments on the wrong line. That is not an accident — it is the point of the exercise.

PART 12 — FREQUENTLY ASKED

Do we need an LLM to pass? No. A Roslyn-analyzer or pattern-based static layer can cover all three categories deterministically and scores full marks on grounding, since every finding is provably tied to real analysis output.

Can our reviewer call a language model? Ask your organisers for the rule in force at this event. If permitted, prompt it only with the specific diff hunk, not the whole file or repo, and require it to return structured JSON with a line reference — never free-text prose you then have to guess a line number for.

Do we need to handle fork PRs? No, unless you want the stretch credit. Same-repo PRs with the default `GITHUB_TOKEN` are enough to pass and are far lower-risk.

What if a file's patch is truncated by GitHub? Skip it and say so in a comment or log — don't guess at content GitHub didn't give you.

Can we post one big comment instead of inline ones? No — Requirement 3 is inline, line-anchored comments via the Review API. A single PR-level comment doesn't meet the contract, even if the content is otherwise correct.

Our reviewer misses some real issues. Are we finished? No, but don't panic either — category coverage is 10 of 100 marks. A reviewer that's narrow but 100% grounded and correctly anchored, sitting on a clean no-duplicate pipeline, scores well. Report the misses honestly — that explanation earns marks.

Good luck — and build the pipe first.